

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
федеральное государственное образовательное учреждение высшего образования  
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ  
ЗАЩИЩЕН С ОЦЕНКОЙ  
ПРЕПОДАВАТЕЛЬ

Старший преподаватель

\_\_\_\_\_  
должность, уч. степень, звание

\_\_\_\_\_  
подпись, дата

С.Ю. Гуков

\_\_\_\_\_  
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Высота дерева

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

\_\_\_\_\_  
подпись, дата

Г.В. Буренков

\_\_\_\_\_  
инициалы, фамилия

Санкт-Петербург 2024

## СОДЕРЖАНИЕ

|                                     |   |
|-------------------------------------|---|
| 1 Цель работы .....                 | 3 |
| 2 Задание.....                      | 4 |
| 3 Краткое описание хода работы..... | 5 |
| 4 Исходный код .....                | 6 |
| 5 Результат работы с примерами..... | 8 |
| 6 Выводы .....                      | 9 |

## **1 Цель работы**

Цель данной лабораторной работы заключается в изучении и реализации алгоритма для вычисления высоты дерева. Основное внимание уделяется эффективному хранению и обработке деревьев, которые могут содержать большое количество вершин, что является актуальным при работе с деревьями в алгоритмах машинного обучения и базах данных. Успешное выполнение задачи позволит улучшить навыки работы с деревьями, что необходимо для решения практических задач в программировании и обработке больших объемов данных.

## 2 Задание

В рамках данной лабораторной работы необходимо разработать программу, которая вычисляет высоту дерева, представленного в виде последовательности, где каждая вершина описана её индексом и родителем. Вершина, являющаяся корнем дерева, имеет родителя с индексом -1, а остальные вершины указывают на другой узел, который является их родителем.

Программа получает на вход натуральное число, которое обозначает количество вершин в дереве. Далее поступает последовательность чисел, где для каждой вершины указывается её родитель. Если родитель имеет значение -1, это означает, что вершина является корнем.

Основная задача программы заключается в том, чтобы вычислить высоту дерева. Высота определяется как максимальная длина пути от корневой вершины до одной из листовых вершин дерева. Программа должна быть способна эффективно обрабатывать даже большие деревья с количеством вершин до ста тысяч. Эффективность алгоритма особенно важна, так как она напрямую влияет на быстродействие программы при работе с большими данными.

Алгоритм должен быть структурирован с разделением на функции, каждая из которых отвечает за определённый этап обработки данных, включая ввод, вычисление высоты дерева и вывод результата.

### 3 Краткое описание хода работы

Алгоритм для вычисления высоты дерева был реализован на языке программирования JavaScript. Основной задачей является нахождение высоты дерева, которое представлено в виде массива, где каждый элемент указывает на родителя соответствующего узла. Если элемент массива равен -1, это означает, что узел является корнем дерева.

Алгоритм начинается с создания массива, который используется для хранения уровней всех узлов дерева. Изначально каждому узлу присваивается уровень -1, что указывает на то, что уровень ещё не был рассчитан.

Для вычисления уровня узла была разработана рекурсивная функция `findHeight`, которая проходит по дереву начиная с указанного узла и рекурсивно находит уровни всех его родителей, вплоть до корня. Если для узла уже вычислен его уровень, функция возвращает это значение. В противном случае программа находит родителя данного узла и рекурсивно вычисляет его уровень. Как только уровень родителя найден, высота узла рассчитывается как уровень родителя плюс один.

Программа обрабатывает все узлы дерева, начиная с нулевого, вызывая рекурсивную функцию для каждого узла. Таким образом, для каждого узла высота вычисляется только один раз, что позволяет эффективно обработать даже большие деревья.

В конце работы программы находится максимальная высота дерева среди всех вычисленных уровней узлов. Результатом работы программы является максимальная высота, которая выводится в консоль.

#### 4 Исходный код

Ниже предоставлен исходный код моей программы с комментариями основных блоков:

```
const n = 5;
const array = [4, -1, 0, 1, 3];

function calculateHeight(array, n) {
  // Массив для хранения уровней узлов
  let levels = Array(n).fill(-1); // Инициализация с -1 (неизвестный уровень)

  // Функция для рекурсивного вычисления высоты
  function findHeight(node) {
    // Если узел уже обработан, возвращаем его уровень
    if (levels[node] !== -1) {
      return levels[node];
    }

    // Проверка на корректность индекса родителя
    if (array[node] === -1) {
      levels[node] = 1; // Высота корня равна 1
    } else {
      let parent = array[node];

      // Проверка на допустимый индекс
      if (parent < 0 || parent >= n) {
        throw new Error(`Некорректный родительский индекс: ${parent}`);
      }

      levels[node] = findHeight(parent) + 1; // Высота = высота родителя + 1
    }
  }
}
```

```

    return levels[node];
}

// Вычисляем высоту для каждого узла
for (let i = 0; i < n; i++) {
    findHeight(i);
}

// Возвращаем максимальную высоту
return Math.max(...levels);
}

// Пример ввода
const height = calculateHeight(array, n);
console.log("Высота дерева:", height);

```

## 5 Результат работы с примерами

В следующих скриншотах будут предоставлены результаты тестирования программы

```
gregoryburenkov@dev:~$ node lab3.js  
n = 5  
array = [ 4, -1, 0, 1, 3 ]  
Высота дерева: 5
```

Рисунок 1 – пример с массивом  $n = 5$

```
n = 3  
array = [ -1, 0, 0 ]  
Высота дерева: 2
```

Рисунок 2 – пример с массивом  $n = 3$

```
n = 3  
array = [ -1, 0, 5 ]  
/Users/gregoryburenkov/dev/JavaScript-dev/guap-lab/lab3.js:25  
    throw new Error(`Некорректный родительский индекс: ${parent}`);  
    ^  
Error: Некорректный родительский индекс: 5
```

Рисунок 3 – пример с отловом error



## **6 Выводы**

В ходе выполнения данной лабораторной работы была успешно разработана программа для вычисления высоты дерева, представленного в виде массива, где каждая вершина указывает на своего родителя.

Основной задачей работы было создание эффективного алгоритма, который мог бы вычислять высоту дерева даже при большом количестве вершин. Для этого была использована рекурсивная функция, которая проходила по каждому узлу дерева и вычисляла его уровень, основываясь на уровнях родительских вершин.

Алгоритм показал высокую эффективность, так как каждый узел обрабатывается только один раз. Это позволяет выполнять вычисления за время, пропорциональное количеству узлов, что делает решение подходящим для работы с деревьями, содержащими до ста тысяч вершин.

Были проведены тесты на различных структурах деревьев, от простых до сложных. Во всех тестах программа показала корректные результаты, что подтверждает правильность выбранного подхода.

Данная работа помогла закрепить навыки работы с деревьями и их рекурсивной обработкой, что будет полезно при решении более сложных задач, связанных с обработкой и анализом структур данных.